

O PEIRAO

Memoria del Proyecto Intermodular

Mateo Varela García

Desarrollo de Aplicaciones Web — 2º Curso

Centro de Formación Profesional Afundación

Junio 2026 · Versión 2.0 (Híbrida con IA)

<https://github.com/Mateo19vg/Proyecto.Peirao.git>

Stack tecnológico: Django · Django REST Framework · React · Tailwind CSS · Open-Meteo
API · Ollama (Llama 3) · Chroma DB

Índice de contenidos

1.	Descripción del proyecto y ámbito de implantación	3
1.1	¿Qué es O Peirao?	3
1.2	Tecnologías empleadas	3
1.3	Usuarios objetivo	3
1.4	Ámbito de implantación	3
1.5	Evolución futura prevista	3
2.	Temporalización del proyecto y fases de desarrollo	4
2.1	Planificación general	4
2.2	Diagrama de Gantt	4
2.3	Diagrama PERT	4
2.4	Retos y aprendizajes	4
3.	Requisitos hardware y software	5
3.1	Requisitos del servidor / máquina de desarrollo	5
3.2	Requisitos del cliente (navegador)	5
3.3	Requisitos software del servidor	5
4.	Arquitectura de la aplicación	6
4.1	Arquitectura general — Tipo 3 (React + API REST Django)	6
4.2	Estructura de ficheros	6
4.3	Diagrama de componentes React	6
4.4	Diagrama de secuencia — Flujo de predicción	6
4.5	Pantallas de la aplicación	7
4.6	Diseño responsivo	7
4.7	Especificación OpenAPI de la fachada REST	7
5.	Descripción de datos y fragmentos de código relevantes	8
5.1	Diagrama SQL — Modelos Django	8
5.2	Algoritmo de predicción — services.py	8
5.3	Llamadas a Open-Meteo — dos APIs simultáneas	8

5.4	Gestión de estado en Log.jsx — filtros y paginación	9
5.5	Envío condicional multipart/JSON en AddCaptura.jsx	9
6.	Conclusiones	10
7.	Referencias y recursos	10

1. Descripción del proyecto y ámbito de implantación

1.1 ¿Qué es O Peirao?

O Peirao ("El Muelle" en gallego) es una aplicación web full-stack orientada a aficionados a la pesca deportiva. Su nombre evoca el punto de partida de toda jornada de pesca: el muelle desde el que se sale a faenar. La aplicación permite a los usuarios:

- Consultar predicciones de idoneidad para pescar una especie concreta en un lugar específico, combinando datos meteorológicos en tiempo real con los rangos óptimos de cada especie.
- Llevar un diario digital de capturas, con campos como especie, spot, fecha, hora, peso, longitud, notas y foto.
- Gestionar un catálogo de especies de pez y lugares de pesca (spots), con coordenadas GPS.

La aplicación responde a una necesidad real: los pescadores deportivos toman decisiones sobre cuándo y dónde salir basándose en su experiencia o en aplicaciones meteorológicas genéricas. O Peirao unifica ambas fuentes de información — condiciones del tiempo y conocimiento sobre la especie objetivo — en una única herramienta personalizada.

1.2 Tecnologías empleadas

Capa	Tecnología	Función principal
Backend	Django REST Framework	API REST, modelos, lógica de negocio
Frontend	React + Vite + Tailwind CSS	Interfaz de usuario, consumo de la API
Base de datos	SQLite (producción: PostgreSQL)	Persistencia de datos
API externa	Open-Meteo (gratuita, sin clave)	Datos meteorológicos en tiempo real
Control de versiones	Git / GitLab	Historial de cambios, entregas periódicas
Inteligencia Artificial	Ollama (Llama 3)	Motor de lenguaje local (LLM) privado para el asistente inteligente
Base de datos vectorial	Chroma DB + LangChain	RAG (Generación Aumentada por Recuperación) para manuales de pesca de Galicia
Mapas y	Leaflet + Geolocalización API	Carga automática de clima real

Capa	Tecnología	Función principal
Geolocalización		en base a coordenadas y mapa interactivo

1.3 Usuarios objetivo

El público principal es el pescador deportivo aficionado con acceso a un dispositivo con navegador web. La interfaz está diseñada para ser intuitiva y usable desde móvil, tablet o escritorio gracias al diseño responsivo con Tailwind CSS.

1.4 Ámbito de implantación

En su versión actual, la aplicación se despliega en modo local (localhost) con fines académicos. La arquitectura desacoplada backend/frontend y el uso de SQLite permiten una migración sencilla a un servidor en producción (por ejemplo, con PostgreSQL, Unicorn y Nginx) sin modificar el código de la aplicación.

1.5 Evolución futura prevista

- [Implementado] Sistema de autenticación con cuentas de usuario y perfiles públicos/privados (para visualización condicional de capturas de terceros).
- Historial de predicciones para comparar condiciones pasadas con capturas registradas.
- [Implementado] Soporte para más variables meteorológicas: fase lunar y cálculo dinámico de mareas en base a coordenadas geográficas.
- Aplicación móvil nativa (React Native) reutilizando la misma API REST.

[Implementado] Asistente inteligente conversacional (SharkAI) integrado con LLM local Ollama (Llama 3), RAG sobre manuales gallegos y corteza de seguridad anti-temas irrelevantes.

2. Temporalización del proyecto y fases de desarrollo

2.1 Planificación general

El proyecto se planificó en 50 horas distribuidas a lo largo de aproximadamente 5 semanas, compaginando su desarrollo con las clases del ciclo. A continuación se detallan las fases principales:

Fase	Actividad	Horas	Semana
F1	Análisis y diseño: definición de modelos, wireframes, elección de tecnologías	6h	1
F2	Configuración del entorno: Django, React, Vite, Tailwind, DRF, proxy	4h	1
F3	Desarrollo del backend: modelos, serializers, endpoints CRUD de especies y spots	8h	2
F4	Integración Open-Meteo: services.py, llamadas a API marina y atmosférica	6h	2-3
F5	Algoritmo de puntuación y endpoint /api/prediccion/	4h	3
F6	Desarrollo del frontend: páginas Home, Predictor, Log, AddCaptura	12h	3-4
F7	Funcionalidades avanzadas: paginación, filtros, preview de foto, multipart	5h	4
F8	Pruebas, corrección de errores y ajustes de UX	3h	5
F9	Redacción de la memoria y README	2h	5
	TOTAL	50h	5 semanas

2.2 Diagrama de Gantt

Diagrama de Gantt — Semanas 1 a 5 (cada columna representa una semana del ciclo de desarrollo)

Fase / Actividad	S1	S2	S3	S4	S5
F1 — Análisis y diseño	■■■■				
F2 — Configuración entorno	■■■■				
F3 — Backend CRUD		■■■■			
F4 — Integración Open-Meteo		■■	■■		
F5 — Algoritmo predicción			■■■■		
F6 — Frontend (4 páginas)			■■	■■	
F7 — Funcionalidades avanzadas				■■■■	
F8 — Testing y corrección					■■
F9 — Memoria y README					■■

2.3 Diagrama PERT

Diagrama PERT — dependencias, tiempos característicos (ES/EF/LS/LF), holgura y ruta crítica.

Tabla de precedencias PERT:

Fase	Actividad	Dur. (h)	Predec.	ES	EF	Holgura
F1	Análisis y diseño	6	—	0	6	0
F2	Configuración entorno	4	F1	6	10	0
F3	Backend CRUD	8	F2	10	18	0
F4	Integración Open-Meteo	6	F3	18	24	0
F5	Algoritmo predicción	4	F4	24	28	0

Fase	Actividad	Dur. (h)	Predec.	ES	EF	Holgura
F6	Frontend (4 páginas)	12	F2, F3, F5	28	40	0
F7	Funcionalidades avanz.	5	F6	40	45	0
F8	Pruebas y correcciones	3	F3, F5, F6, F7	45	48	0
F9	Memoria y README	2	F8	48	50	0

Ruta crítica: F1 → F2 → F3 → F4 → F5 → F6 → F7 → F8 → F9 (duración total: 50 horas). Todas las fases tienen holgura cero, lo que significa que cualquier retraso en cualquier fase afecta directamente a la fecha de entrega. El camino más corto alternativo (F1 → F2 → F3 → F8 → F9) solo dura 23 horas, muy por debajo de la ruta crítica.

2.4 Retos encontrados

A continuación se describen los principales obstáculos técnicos encontrados durante el desarrollo y cómo se superaron:

Integración de Open-Meteo: La API marina y la atmosférica tienen estructuras de respuesta distintas y endpoints separados. Fue necesario leer con detenimiento la documentación, gestionar los casos en que la API marina no devuelve datos (zonas de interior sin costa) y combinar ambas respuestas en un único diccionario Python antes de pasarlo al algoritmo de puntuación.

Proxy de Vite y CORS: Configurar el proxy para que el frontend en el puerto 5173 se comunicara con el backend Django en el puerto 8000 sin errores CORS fue el primer obstáculo real del proyecto. La solución con vite.config.js resuelve el problema en desarrollo, pero requirió entender la diferencia entre el entorno de desarrollo (proxy transparente) y el de producción (donde se necesitaría Nginx o un CORS configurado explícitamente).

Formulario multipart con foto: El endpoint de creación de capturas acepta tanto JSON como multipart/form-data dependiendo de si el usuario adjunta una foto. Implementar la lógica condicional en el frontend (detectar si form.foto es un objeto File o null) y en el serializer de Django (donde ImageField procesa correctamente ambos formatos) supuso un reto inesperado que requirió varias iteraciones de prueba y error.

Paginación y filtros combinados: Mantener el estado de los filtros activos al cambiar de página, y resetear automáticamente a la página 1 al modificar cualquier filtro, requirió una

gestión de estado cuidadosa con `useState` y `useEffect` en React. La solución final usa dos `useEffect` independientes: uno que observa `[pagina, filtros]` y lanza la llamada a la API, y otro que observa solo `[filtros]` y resetea la página a 1.

2.5 Aprendizajes obtenidos

Los retos anteriores generaron los siguientes aprendizajes duraderos que se aplicarán en futuros proyectos:

Diseño de APIs externas heterogéneas: Trabajar con dos APIs con formatos de respuesta distintos enseñó a abstraer la capa de comunicación en un módulo independiente (`services.py`), separando la lógica de negocio del acceso a datos externos. Este patrón de código es directamente exportable a cualquier proyecto que consuma APIs de terceros.

Arquitectura desacoplada y entornos de desarrollo/producción: Entender la diferencia entre el proxy de Vite (solo útil en desarrollo) y la configuración CORS real (necesaria en producción) consolidó la comprensión de cómo funciona una arquitectura frontend/backend separada en los distintos entornos de despliegue.

Gestión de formularios complejos en React: El formulario multipart enseñó que no todos los datos son texto plano: los archivos binarios requieren un tratamiento distinto tanto en el cliente como en el servidor. Se aprendió a usar `FormData` y a detectar dinámicamente el `Content-Type` adecuado según el tipo de dato.

Estado derivado y efectos reactivos en React: La paginación con filtros consolidó el uso correcto de `useEffect` con dependencias precisas. Aprender que el estado de la página es estado derivado de los filtros (y por tanto debe resetearse cuando estos cambian) fue una lección importante sobre el flujo de datos unidireccional de React.

Diseño responsivo con Tailwind CSS: Aprender la filosofía mobile-first de Tailwind desde cero resultó muy productivo. Las clases responsivas (`sm:`, `md:`, `lg:`) permiten definir el diseño adaptativo directamente en el HTML, sin necesidad de media queries CSS separadas, lo que hace el código más limpio y mantenible.

3. Requisitos hardware y software

3.1 Requisitos del servidor / máquina de desarrollo

Componente	Mínimo	Recomendado
Procesador	Dual-core 1.5 GHz (x64)	Quad-core 2.5 GHz o superior
Memoria RAM	4 GB	8 GB o más
Espacio en disco	500 MB libres	2 GB (para entorno virtual, node_modules, media)
Sistema operativo	Ubuntu 20.04 / Windows 10 / macOS 12	Ubuntu 22.04 LTS / macOS 14
Tarjeta gráfica	Integrada (solo renderizado 2D)	Dedicada (mayor fluidez en mapas Leaflet)
Conexión a Internet	Requerida (Open-Meteo API)	Banda ancha $\geq$ 10 Mbps

3.2 Requisitos del cliente (navegador)

Navegador	Versión mínima	Soporte
Google Chrome	110	Completo
Mozilla Firefox	110	Completo
Microsoft Edge	110	Completo
Safari	16	Completo
Internet Explorer	Cualquier versión	No soportado

3.3 Requisitos software del servidor

Componente	Versión	Propósito
Python	3.11	Lenguaje del backend
Django	5.0	Framework web backend
Django REST Framework	3.15	Construcción de la API REST

Componente	Versión	Propósito
Pillow	10	Procesamiento de imágenes (fotos de capturas)
Requests	2.31	Llamadas HTTP a Open-Meteo desde el backend
Node.js	20 LTS	Entorno de ejecución para el frontend
npm	10	Gestión de dependencias del frontend
React	18	Biblioteca de interfaz de usuario
Vite	5	Bundler y servidor de desarrollo del frontend
Tailwind CSS	3	Framework de estilos utility-first
Axios	1.6	Cliente HTTP para peticiones del frontend
React Router DOM	6	Enrutamiento en el frontend (SPA)
Git	2.40	Control de versiones
Visual Studio Code	1.88	Editor de código (backend y frontend)
Servidor (desarrollo)	localhost	Django dev server (8000) + Vite (5173)

Ollama	0.1.x superior	o Gestor local de modelos LLM privados
Llama 3 (8B)	8B parámetros	Modelo fundacional de IA para el chat del asistente
Chroma DB	0.4.x superior	o Base de datos vectorial para manuales de pesca
langchain	0.1.x superior	o Integración del flujo RAG y embeddings del manual

4. Arquitectura de la aplicación

O Peirao sigue la modalidad Tipo 3 del proyecto intermodular: un frontend en React que consume una API REST construida con Django. Ambos proyectos son independientes y se comunican exclusivamente a través de HTTP. Adicionalmente, el backend se conecta con un motor local de IA (Ollama con modelo llama3) y una base de datos vectorial Chroma DB para dar soporte al asistente inteligente conversacional.

O Peirao sigue la modalidad Tipo 3 del proyecto intermodular: un frontend en React que consume una API REST construida con Django. Ambos proyectos son independientes y se comunican exclusivamente a través de HTTP.

Flujo de comunicación: Navegador (React, puerto 5173) → proxy Vite → Django DRF (puerto 8000) → SQLite / Open-Meteo API

4.2 Estructura de ficheros

Backend (Django)

Archivo / Carpeta	Responsabilidad
manage.py	Comando principal de Django
requirements.txt	Dependencias Python del proyecto
core/settings.py	Configuración global: INSTALLED_APPS, MEDIA_ROOT, REST_FRAMEWORK
core/urls.py	Raíz del enrutamiento: incluye api.urls y sirve /media/
api/models.py	Definición de las tablas: Especie, Spot, Captura
api/serializers.py	Conversión modelo ↔ JSON para la API REST
api/views.py	Lógica de cada endpoint (ViewSet + APIView)
api/services.py	Llamadas a Open-Meteo + algoritmo de puntuación
api/urls.py	Enrutamiento de la API con DefaultRouter
api/fixtures/initial_data.json	Datos de ejemplo para carga inicial

Frontend (React + Vite)

Archivo / Carpeta	Responsabilidad
vite.config.js	Proxy de desarrollo: /api/* → http://localhost:8000
src/App.jsx	Configuración de React Router: define las 4 rutas
src/services/api.js	Centraliza todas las llamadas HTTP con Axios
src/components/Navbar.jsx	Barra de navegación compartida entre páginas
src/pages/Home.jsx	Página de inicio con tarjetas de navegación
src/pages/Predictor.jsx	Formulario de predicción + visualización del resultado
src/pages/Log.jsx	Historial de capturas con filtros y paginación
src/pages/AddCaptura.jsx	Formulario para registrar una nueva captura
src/components/MapPicker.jsx	Mapa interactivo con capas batimétricas para selección de coordenadas de pesca
src/pages/Chat.jsx	Página del chat del asistente de pesca inteligente (SharkAI) con modo clima automatizado

4.3 Diagrama de componentes React

Jerarquía de componentes React por página. Los estados principales (●) se indican junto al componente que los gestiona.

Página / Componente	Estado principal (●)	Props recibidas
App.jsx	— (solo rutas)	—
└─ Navbar.jsx	—	—
└─ Home.jsx	—	—
└─ Predictor.jsx ●	spotId, especieId, prediccion, loading	—
└─ (select spot)	—	spots[], spotId, onChange
└─ (select especie)	—	especies[], especieId,

Página / Componente	Estado principal (●)	Props recibidas
		onChange
└─ (ResultadoCard)	—	prediccion
└─ Log.jsx ●	capturas, filtros, pagina, total	—
└─ (FiltroBar)	—	filtros, onChange
└─ (CapturaCard)	—	captura, onDelete
└─ (Paginacion)	—	pagina, total, onChange
└─ AddCaptura.jsx ●	form, preview, loading	—
└─ (ImagePreview)	—	src

4.4 Diagrama de secuencia — Flujo de predicción

Diagrama de secuencia: el usuario solicita una predicción de pesca desde la página Predictor.jsx

Pas o	Actor	Acción / Evento
1	Usuario	Selecciona un Spot y una Especie en los desplegables
2	Usuario	Pulsa el botón 'Ver predicción'
3	Predictor.jsx	onClick → llama a api.getPrediccion(spotId, especieId)
4	api.js (Axios)	GET /api/prediccion/?spot_id=X&especie_id=Y → proxy Vite → Django
5	api/views.py	PrediccionView.get() → extrae parámetros del query string
6	api/services.py	calcular_prediccion() → llama a Open-Meteo (API atmosférica + marina)
7	Open-Meteo	Devuelve temperatura del aire, viento, temperatura del mar, oleaje
8	api/services.py	calcular_puntuacion() → compara con rangos óptimos de la especie
9	api/views.py	Serializa y devuelve JSON con condiciones + puntuación + resumen

Pas o	Actor	Acción / Evento
10	Predictor.jsx	useState(prediccion) actualizado → React re-renderiza
11	ResultadoCard	Muestra barra de puntuación y tarjetas de datos meteorológicos

4.5 Pantallas de la aplicación

La aplicación consta de 4 páginas principales, accesibles desde la barra de navegación:

Página	Ruta	Descripción
Home	/	Página de inicio con tarjetas de acceso rápido a Predictor y Log
Predictor	/predictor	Selector de spot + especie → predicción en tiempo real con puntuación visual
Log de capturas	/log	Tabla paginada de capturas con filtros por especie, spot y búsqueda de texto
Añadir captura	/add	Formulario con validación, preview de foto y envío multipart/JSON
Asistente SharkAI	/chat	Chat conversacional con IA híbrida, clima real automatizado por localización y RAG

4.6 Diseño responsivo

La interfaz se ha diseñado con la filosofía mobile-first de Tailwind CSS. Las clases responsivas (sm:, md:, lg:) garantizan que el layout se adapta a cualquier dispositivo:

- Móvil (< 640px): columna única, botones de ancho completo, tabla de capturas en tarjetas apiladas.
- Tablet (640–1024px): grid de 2 columnas en la página Home y en los formularios.
- Escritorio (> 1024px): layout de 3 columnas en Home, tabla clásica en Log con todas las columnas visibles.

No se usan media queries CSS manuales: toda la adaptación se gestiona con las utilidades responsivas de Tailwind, lo que hace el código más limpio y mantenible.

4.7 Especificación OpenAPI de la fachada REST

Explorable en tiempo real en <http://localhost:8000/api/> (DRF Browsable API)

Método	Endpoint	Tipo de parámetro	Descripción
GET	/api/especies/	—	Lista todas las especies
POST	/api/especies/	Body JSON	<code>{"nombre":"Lubina","temp_agua_min":12,"temp_agua_max":19,"viento_max":25}</code>
GET	/api/especies/{id}/	Path param: id	Detalle de una especie
PUT	/api/especies/{id}/	Path param + Body JSON	Actualización completa
DELETE	/api/especies/{id}/	Path param: id	Elimina la especie
GET	/api/spots/	—	Lista todos los spots
POST	/api/spots/	Body JSON	<code>{"nombre":"Baiona P-1","tipo":"mar","latitud":42.12,"longitud":-8.84}</code>
GET	/api/spots/{id}/	Path param: id	Detalle de un spot
PUT	/api/spots/{id}/	Path param + Body JSON	Actualización completa
DELETE	/api/spots/{id}/	Path param: id	Elimina el spot
GET	/api/capturas/	Query params opcionales	Lista paginada (10/pág), filtros por especie, spot, search
POST	/api/capturas/	Body multipart o JSON	multipart/form-data (con foto): campos especie, spot, fecha, hora + foto (File). JSON (sin foto): <code>{"especie":1,"spot":5,"fecha":"2025-06-01","hora":"08:30"}</code>
DELETE	/api/capturas/{id}/	Path param: id	Elimina la captura
GET	/api/prediccion/	Query params: spot_id, especie_id	Predicción de idoneidad de pesca en tiempo real

5. Descripción de datos y fragmentos de código relevantes

5.1 Diagrama SQL — Modelos Django

La base de datos tiene 3 tablas. Captura relaciona Especie y Spot con claves foráneas (relación N:1 desde Captura hacia cada una).

Tabla	Campo	Tipo Python/SQL	Restricciones
Especie	id	AutoField / INTEGER PK	AUTO_INCREMENT
	nombre	CharField(100) / VARCHAR(100)	UNIQUE, NOT NULL
	temp_agua_min	FloatField / REAL	NOT NULL
	temp_agua_max	FloatField / REAL	NOT NULL
	viento_max	FloatField / REAL	NOT NULL
	descripcion	TextField / TEXT	NULLABLE
Spot	id	AutoField / INTEGER PK	AUTO_INCREMENT
	nombre	CharField(100) / VARCHAR(100)	NOT NULL
	tipo	CharField(20) / VARCHAR(20)	Choices: mar/río/embalse/lago
	latitud	FloatField / REAL	NOT NULL
	longitud	FloatField / REAL	NOT NULL
	descripcion	TextField / TEXT	NULLABLE
Captura	id	AutoField / INTEGER PK	AUTO_INCREMENT
	especie_id	ForeignKey → Especie	ON DELETE CASCADE
	spot_id	ForeignKey → Spot	ON DELETE CASCADE
	fecha	DateField / DATE	NOT NULL
	hora	TimeField / TIME	NOT NULL

Tabla	Campo	Tipo Python/SQL	Restricciones
	peso_kg	FloatField / REAL	NULLABLE
	longitud_cm	FloatField / REAL	NULLABLE
	temperatura_agua	FloatField / REAL	NULLABLE
	velocidad_viento	FloatField / REAL	NULLABLE
	notas	TextField / TEXT	NULLABLE
	foto	ImageField VARCHAR(100)	NULLABLE, guardado en media/capturas/

5.2 Algoritmo de predicción — services.py

El fragmento más representativo del backend es la función `calcular_puntuacion()`. Recibe un objeto especie (con sus rangos óptimos) y un diccionario condiciones (obtenido de Open-Meteo). El tipo de retorno es un entero entre 0 y 100.

```
Pseudocódigo de calcular_puntuacion(especie, condiciones):
puntuacion ← 100
temp_agua ← condiciones["temperatura_agua"] # float, en °C
viento ← condiciones["velocidad_viento"] # float, en km/h
SI temp_agua < especie.temp_agua_min:
    diferencia ← especie.temp_agua_min - temp_agua # float
    penalizacion ← min(diferencia * 4, 40) # máximo 40 puntos
puntuacion ← puntuacion - penalizacion
SI temp_agua > especie.temp_agua_max:
    diferencia ← temp_agua - especie.temp_agua_max
    penalizacion ← min(diferencia * 4, 40)
puntuacion ← puntuacion - penalizacion
SI viento > especie.viento_max:
    exceso ← viento - especie.viento_max
    penalizacion ← min(exceso * 2, 40)
puntuacion ← puntuacion - penalizacion
RETORNAR max(puntuacion, 0) # nunca negativo
```

Análisis de tipos y flujo:

- `temp_agua` y `viento` son float (número decimal). La diferencia entre ellos y los umbrales de la especie también es float.
- `penalizacion` se calcula como float pero se recorta a un máximo de 40 con la función `min()` nativa de Python.
- La función `max(puntuacion, 0)` al final garantiza que el resultado nunca sea negativo (tipo int o float, siempre ≥ 0).
- El algoritmo es lineal: no hay bucles, solo tres condicionales independientes (if/elif no aplica, pues temperatura y viento son penalizaciones acumulativas).

5.3 Llamadas a Open-Meteo — dos APIs simultáneas

La función `obtener_condiciones(spot)` en `services.py` hace dos peticiones HTTP a APIs distintas y combina los resultados en un único diccionario Python:

Flujo de obtener_condiciones(spot): 1. Construir URL atmosférica con latitud y longitud del spot → GET `api.open-meteo.com/v1/forecast?latitude=X&longitude=Y` → Parámetros: `temperature_2m`, `wind_speed_10m`, `weathercode` → Tipo respuesta: dict JSON con clave 'current_weather' 2. Construir URL marina con las mismas coordenadas → GET `marine-api.open-meteo.com/v1/marine?latitude=X&longitude=Y` → Parámetros: `sea_surface_temperature`, `wave_height` → Tipo respuesta: dict JSON con clave 'hourly' 3. Extraer valores del momento actual: `temperatura_aire` ← float (puede ser None si la API falla) `velocidad_viento` ← float (puede ser None) `temperatura_agua` ← float (None en spots de interior sin costa) `altura_olas` ← float (None en spots sin datos marinos) 4. Retornar dict con los 4 valores → Si `temperatura_agua` es None → el algoritmo usa solo `temp_aire` y viento

La gestión de None es clave: los spots de río o embalse no tienen temperatura marina. El algoritmo de puntuación comprueba si el valor es None antes de aplicar la penalización de temperatura del agua.

5.4 Gestión de estado en Log.jsx — filtros y paginación

La página `Log.jsx` es la más compleja del frontend en cuanto a gestión de estado. Tiene 4 estados interrelacionados:

- `capturas`: array de objetos (tipo: `Captura[]`) → la lista que se muestra en pantalla.
- `pagina`: número entero (int) → página actual de la paginación, empieza en 1.
- `total`: número entero → total de capturas que cumplen los filtros (devuelto por la API en el campo `count`).
- `filtros`: objeto `{especie: string, spot: string, search: string}` → los filtros activos.

Pseudocódigo del useEffect principal de Log.jsx: `useEffect` ejecutar cuando [`pagina`, `filtros`] cambien: `params` ← {} # objeto vacío SI `filtros.especie !== ""`: `params.especie` ← `filtros.especie` # string → int en el servidor SI `filtros.spot !== ""`: `params.spot` ← `filtros.spot` SI `filtros.search !== ""`: `params.search` ← `filtros.search` `params.page` ← `pagina` # int `respuesta` ← `AWAIT getCapturas(params)` # llamada async a Axios `capturas` ← `respuesta.results` # array de objetos `total` ← `respuesta.count` # int Cuando `filtros` cambian → resetear `pagina` a 1 (`useEffect` separado)

El reset de página al cambiar filtros se gestiona en un `useEffect` independiente que observa solo filtros. Cuando el usuario cambia un filtro, este efecto corre primero y establece `pagina = 1`, lo que a su vez dispara el efecto principal con los nuevos parámetros.

5.5 Envío condicional multipart/JSON en AddCaptura.jsx

El formulario de añadir captura cambia el formato de la petición HTTP según si el usuario ha adjuntado una foto o no. Esto se gestiona con la siguiente lógica:

```
Pseudocódigo de handleSubmit en AddCaptura.jsx: SI form.foto ES un objeto File: data ← new  
FormData() # objeto binario PARA CADA campo EN form: data.append(campo,  
valor) # incluye el File en binario Content-Type ← 'multipart/form-data' SI NO: data ←  
JSON.stringify(form) # string JSON Content-Type ← 'application/json' respuesta ← AWAIT  
createCaptura(data) SI respuesta.ok: navigate('/log') # redirige a la lista
```

En el backend (serializer de Django), el campo foto usa ImageField, que DRF procesa correctamente en ambos formatos. Si no se envía foto, el campo queda como null en la base de datos y el frontend muestra un icono genérico en su lugar.

6. Conclusiones

O Peirao ha sido el proyecto más completo desarrollado durante el ciclo formativo. Ha supuesto integrar por primera vez en un único trabajo todas las capas de una aplicación web moderna: diseño de base de datos, API REST, lógica de negocio, consumo de APIs externas y frontend interactivo.

Los objetivos iniciales se han cumplido: la aplicación es funcional, responsiva y desplegable. El mayor aprendizaje ha sido entender cómo las piezas encajan en la arquitectura desacoplada frontend/backend, y cómo la gestión de estado en React determina la experiencia del usuario.

El nombre O Peirao tiene también un significado personal: en Galicia, el muelle es el punto de encuentro entre el pescador y el mar. Esta aplicación pretende ser ese punto de encuentro entre los datos y la intuición del pescador.

7. Referencias y recursos

- Documentación oficial de Django: <https://docs.djangoproject.com/>
- Django REST Framework: <https://www.django-rest-framework.org/>
- React — Documentación oficial: <https://react.dev/>
- Vite: <https://vitejs.dev/>
- Tailwind CSS: <https://tailwindcss.com/docs/>
- Open-Meteo API (gratuita, sin clave): <https://open-meteo.com/>
- Open-Meteo Marine API: <https://open-meteo.com/en/docs/marine-weather-api>
- Axios: <https://axios-http.com/>
- React Router v6: <https://reactrouter.com/>

5.6 Integración del asistente de chat inteligente (SharkAI) y RAG

La funcionalidad más avanzada e innovadora incorporada al proyecto es SharkAI, un asistente inteligente conversacional privado que reside localmente. El asistente no solo mantiene una conversación con estilo marinero y retranca gallega, sino que toma las decisiones basándose en datos reales mediante un sistema RAG (Retrieval-Augmented Generation) y los cálculos meteorológicos locales de Galicia.

Lógica y flujo de ejecución en `views.py`:

1. Recepción de parámetros: El frontend envía la pregunta del pescador, la especie objetivo, el historial reciente de 4 mensajes y, opcionalmente, la latitud y longitud.
2. Cálculo dinámico de marea: Si se aportan coordenadas, el servidor calcula de manera autónoma el estado de la marea y la fase lunar actual (para calcular el coeficiente de marea) mediante las fórmulas físicas de `services.py`, alimentando el motor de idoneidad.

3. Extracción de contexto RAG: Se realiza una consulta semántica sobre la base de datos vectorial Chroma (que contiene manuales de pesca reales de Galicia) para recuperar extractos sobre la especie o técnicas relevantes.
4. Corteza de seguridad (Anti-Pizza): Antes de enviar la petición al LLM, un filtro de contenido analiza el texto para asegurar que la consulta está estrictamente relacionada con la pesca, rechazando temas irrelevantes de forma educada.
5. Consulta a Ollama (Llama 3): Se construye un prompt unificado estructurado (System Prompt + Historial + Clima real + Contexto RAG) y se envía a la API local de Ollama (<http://127.0.0.1:11434/api/chat>) con un timeout extendido de 120 segundos para inferencia estable en CPU.
6. Fallback resiliente: Si el servidor Ollama no está operativo, el backend intercepta el fallo y devuelve una predicción simulada de alta calidad estructurada en base a las reglas matemáticas de services.py.